Лабораторна робота №3

З дисципліни “Системи програмування”

На тему:“Бібліотека TPL. Розпаралелювання циклів Parallel.For та Parallel.ForEach. Управління станом ітерацій“

Студента 4 курсу: Групи
МІТ-41 Демиденко Андрій

Київ - 2026р.

Завдання 1: Дослідження ефективності розпаралелювання (Parallel.For) залежно від складності обчислень

Мета: Провести порівняльний бенчмаркінг послідовного (`for`) та паралельного (`Parallel.For`) виконання циклів. Дослідити вплив обчислювальної складності ітерації (від простого ділення до комплексних математичних функцій) на коефіцієнт прискорення.

Опис реалізації: Створено метод `RunExperiment`, який обробляє масиви розміром 10 та 50 мільйонів елементів. Для точного вимірювання часу використано клас `Stopwatch`. Задачі поділено на 5 рівнів складності:

1. **`IntDiv10`** (найпростіша операція: цілочисельне ділення).
2. **`DoubleDiv10`** (ділення чисел з плаваючою комою).
3. **`DoubleDivPi`** (ділення на константу `Math.PI`).
4. **`DoubleExpDivPowPi`** (комбінація функцій `Math.Exp` та `Math.Pow`).
5. **`DoubleExpPiDivPowPi`** (високонавантажена математична операція).

Опис роботи та аналіз результатів:

Програма демонструє важливий архітектурний принцип TPL: паралелізм не є безкоштовним.

Для тривіальних операцій (IntDiv10) паралельне виконання може виявитися повільнішим за послідовне (прискорення < 1.0x). Це пов'язано з тим, що час, витрачений процесором на розбиття масиву (partitioning), синхронізацію потоків та управління чергою пулу, перевищує час самого обчислення. Також виникає проблема "вузького місця" пропускної здатності оперативної пам'яті.

Проте, зі зростанням "ваги" ітерації (наприклад, виклик важких методів Math.Exp), накладні витрати на TPL стають непомітними, і ми спостерігаємо лінійне прискорення, яке наближається до кількості фізичних ядер процесора.

Тип/Складність	Розмір	Послідовно (с)	Паралельно (с)	Прискорення
int: x = x / 10	10000000	0.02060	0.02360	0.87x
double: x = x / 10	10000000	0.04828	0.02059	2.35x
double: x = x / PI	10000000	0.05434	0.02442	2.22x
double: x = e^x / x^PI	10000000	0.23808	0.05602	4.25x
double: x = e^(PI*x) / x^PI	10000000	0.22751	0.05801	3.92x
int: x = x / 10	50000000	0.09891	0.05809	1.70x
double: x = x / 10	50000000	0.23048	0.09291	2.48x
double: x = x / PI	50000000	0.23688	0.09495	2.49x
double: x = e^x / x^PI	50000000	1.22782	0.32369	3.79x
double: x = e^(PI*x) / x^PI	50000000	1.13959	0.27561	4.13x

Рисунок №1 — console output Task1.

Завдання 2: Пошук та переривання циклу Parallel.For

Мета: Реалізувати паралельний пошук заданого значення у великому масиві даних з використанням об'єкта `ParallelLoopState` та методу `Break()`.

Опис реалізації: Згенеровано масив типу `double` на 50 млн елементів. Цільове значення (500.0) штучно розміщено на індексах 15_000_000 та 25_000_000. Враховуючи специфіку арифметики з плаваючою комою, порівняння здійснюється не через строгу рівність, а через модуль різниці (`Math.Abs(x - TARGET) <= EPSILON`). Для переривання використовується `state.Break()`.

Опис роботи програми: На відміну від стандартного оператора `break` у звичайному циклі `for`, виклик `state.Break()` у багатопотоковому середовищі не зупиняє цикл миттєво для всіх потоків. Він сигналізує планувальнику TPL, що потоки не повинні починати виконання ітерацій з індексами, *більшими* за поточний. Проте всі ітерації з індексами, *меншими* за той, де було викликано `Break`, гарантовано будуть завершені. Саме тому властивість `loopResult.LowestBreakIteration` є критично важливою: вона гарантує, що ми отримаємо найменший індекс (15 000 000), навіть якщо інший потік першим знайшов значення на індексі 25 000 000.

```
Ініціалізація масиву даних...
Пошук значення в околі числа 500 з відхиленням 0.05...
[Потік 11] Знайдено відповідність на індексі 25000000 (Значення: 500.01). Ініціація Break()...
[Потік 1] Знайдено відповідність на індексі 499966 (Значення: 499.966). Ініціація Break()...
[Потік 15] Знайдено відповідність на індексі 499951 (Значення: 499.951). Ініціація Break()...

--- ЗВІТ ВИКОНАННЯ ---
Статус IsCompleted: False
Найменший індекс переривання (LowestBreakIteration): 499951
Підтверджене значення на індексі переривання: 499.951
```

Рисунок №2 — console output Task2.

Завдання 3: Ітерація по колекціях через Parallel.ForEach із застосуванням делегатів (іменованих методів)

Мета: Дослідити механізм розпаралелювання обробки колекцій за допомогою методу `Parallel.ForEach`. Реалізувати логіку ітерації шляхом передачі іменованого методу (делегата), який взаємодіє з об'єктом управління станом `ParallelLoopState`.

Опис реалізації: Алгоритм виконує паралельний пошук цільового значення (`TARGET_VALUE = 500.0`) у масиві з 50 000 000 елементів типу `double` із допустимою похибкою (`EPSILON = 0.05`). Архітектурною особливістю цієї

реалізації є винесення логіки порівняння в окремий статичний метод `ProcessElement`. Цей метод строго відповідає сигнатурі узагальненого делегата `Action<TSource, ParallelLoopState>`, який вимагається перевантаженням методу `Parallel.ForEach`. Цільові значення для симуляції успішного пошуку штучно розміщено на індексах 20 000 000 (значення 500.04) та 40 000 000 (значення 500.02).

Опис роботи програми (Технічний аналіз):

1. **Партиціювання:** Метод `Parallel.ForEach` автоматично визначає тип вхідної колекції (у даному випадку масив, який реалізує `IEnumerable<double>`) і застосовує внутрішній механізм партиціювання (розбиття масиву на сегменти) для рівномірного розподілу навантаження між потоками пулу (`ThreadPool`).
2. **Обробка стану:** На кожній ітерації планувальник TPL передає в метод `ProcessElement` поточне значення елемента та екземпляр `ParallelLoopState`.
3. **Переривання:** При виконанні математичної умови `Math.Abs(value - TARGET_VALUE) <= EPSILON`, потік викликає `pls.Break()`. Це ініціює кооперативне скасування: потоки завершують поточні ітерації, але нові ітерації з індексами, вищими за індекс переривання, не запускаються.
4. **Специфіка LowestBreakIteration:** Хоча `ForEach` працює з елементами колекції, а не з явними числовими лічильниками, планувальник TPL внутрішньо відстежує порядковий номер елемента в послідовності. `loopResult.LowestBreakIteration` повертає гарантовано найменший 64-бітний індекс (порядковий номер), на якому відбувся виклик `Break()`.

```
Ініціалізація масиву даних...
Пошук значення в околі 500 (відхилення 0.05) за допомогою ForEach...
[Потік 1] Знайдено відповідність. Значення: 499.951. Ініціація Break().

--- ЗВІТ ВИКОНАННЯ ---
Статус IsCompleted: False
Найменша ітерація переривання (LowestBreakIteration): 499951
Підтверджене значення за індексом ітерації 499951: 499.951
```

Рисунок №3 — console output Task3.

Завдання 3: Використання Parallel.ForEach з лямбда-виразами та механізмом лексичного замикання (Closure)

Мета: Модифікувати алгоритм паралельного обходу колекції, замінивши використання глобальних констант та іменованих методів на анонімні функції (лямбда-вирази) з інкапсуляцією стану через лексичне замикання.

Опис реалізації: Вхідні дані та мета алгоритму ідентичні Завданню 3 (масив на

50 млн елементів). Змінено розміщення цільових значень: індекси 12 500 000 (значення 500.04) та 33 000 000 (значення 500.01). Константи `TARGET_VALUE` та `EPSILON` переміщено з рівня класу у локальну область видимості методу `Main`. Логіку обробки реалізовано інлайн у вигляді лямбда-виразу `(currentValue, loopState) => { ... }`, який передається безпосередньо як аргумент у метод `Parallel.ForEach`.

Опис роботи програми (Технічний аналіз):

1. **Лексичне замикання (Closure):** Ключова відмінність полягає в механізмі управління пам'яттю на етапі компіляції. Оскільки лямбда-вираз використовує локальні константи `TARGET_VALUE` та `EPSILON` з методу `Main`, компілятор C# (Roslyn) автоматично генерує прихований клас (closure class). Цей клас захоплює необхідні змінні з контексту методу `Main` і передає їх у потоки пулу.
2. **Безпека та інкапсуляція:** Цей підхід усуває необхідність використання статичних полів класу (як у Завданні 3), забезпечуючи жорстку ізоляцію стану та уникаючи забруднення глобальної області видимості класу.
3. **Виконання:** Робота планувальника TPL, партиціювання та механізм переривання через `loopState.Break()` функціонують ідентично Завданню 3. Запис статусу `IsCompleted` повертає `False`, підтверджуючи, що цикл було перервано достроково.

```
Ініціалізація масиву даних...
Пошук значення в околі 500 (відхилення 0.05) через лямбда-вираз...
[Потік 16] Знайдено відповідність: 500.04. Виклик loopState.Break().
[Потік 1] Знайдено відповідність: 499.951. Виклик loopState.Break().
[Потік 6] Знайдено відповідність: 499.966. Виклик loopState.Break().
[Потік 7] Знайдено відповідність: 499.982. Виклик loopState.Break().

--- ЗВІТ ВИКОНАННЯ ---
Статус IsCompleted: False
Найменша ітерація переривання (LowestBreakIteration): 499951
Підтверджене значення: 499.951
```

Рисунок №4 — console output Task4.

Висновок:

У ході виконання лабораторної роботи №3 було досліджено архітектуру Task Parallel Library (TPL) для реалізації паралелізму на рівні даних (Data Parallelism). За допомогою бенчмаркінгу в Завданні 1 було експериментально доведено, що ефективність `Parallel.For` прямо пропорційно залежить від обчислювальної складності ітерацій (гранулярності задачі): застосування TPL виправдане лише для ресурсомістких алгоритмів.

Виконуючи Завдання 2–4, було опановано механізми кооперативного

переривання асинхронних циклів за допомогою об'єкта `ParallelLoopState`. Використання методу `Break()` дозволило реалізувати детермінований паралельний пошук, що гарантує знаходження елемента з найменшим індексом, імітуючи поведінку послідовного циклу, але використовуючи обчислювальні потужності всіх ядер процесора. Навички використання лямбда-виразів та замикань спростили побудову паралельних конструкцій, зменшивши кількість шаблонного коду.